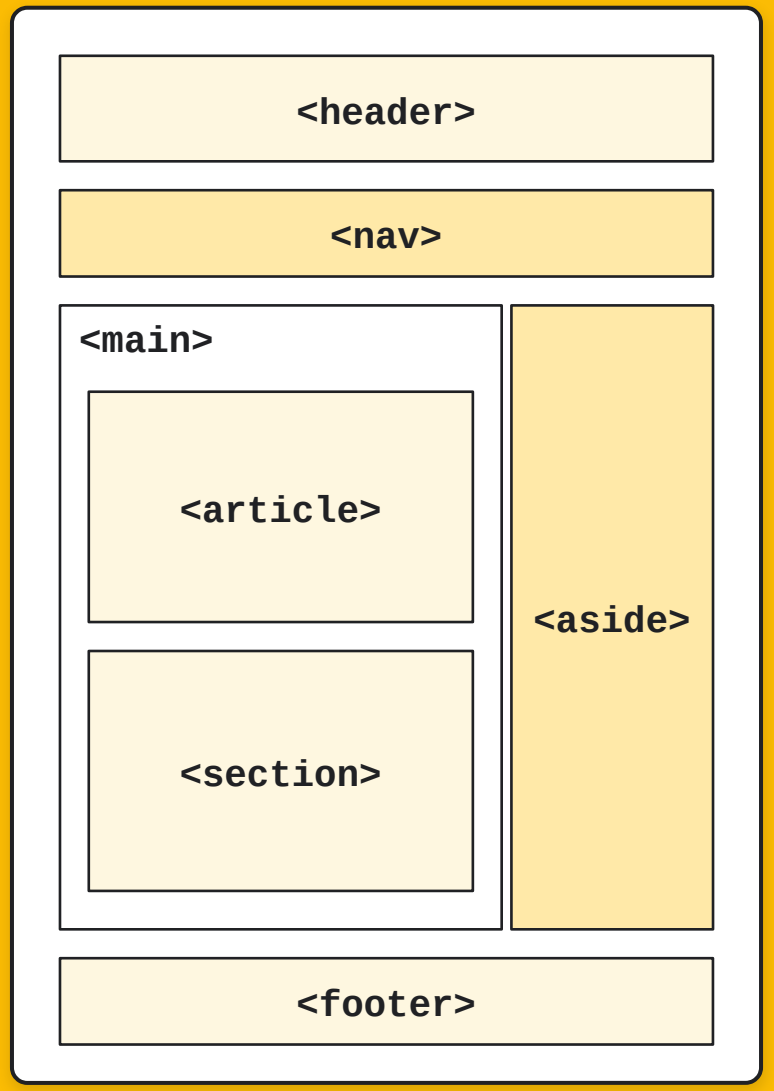
MÓDULO DIW · DAW — UNIDAD DIDÁCTICA 3

HTML5 Semántico, Estructura Accesible y Formularios

RA2 · Estructura semántica y formularios accesibles

- Comprender qué aporta la semántica HTML5 frente a la «divitis»
- Construir un outline de documento correcto y navegable
- Crear formularios accesibles con validación nativa



DÓNDE ESTAMOS

Hoy escribimos la primera línea de código



La pregunta de toda la unidad: no me importa cómo se ve una cosa, me importa qué es.

En la parte de HTML no se escribe ni una línea de CSS. La página va a quedar fea, y eso está bien.

CONTEXTO PROFESIONAL

¿Por qué esto importa en el mundo real?

Lector de pantalla · Puntos de referencia

banner	`<header>`
navegación	`<nav>`
principal	`<main>`
complementario	`<aside>`
información de contenido	`<footer>`

Un no aparece en esta lista.

Accesibilidad legal

Sector público y, desde 2025, también el comercio electrónico

SEO

Google lee el HTML igual que un lector de pantalla

Compatibilidad universal

Funciona en cualquier navegador, sin compilar nada

Puesto de trabajo real

El outline semántico se revisa antes de entregar

Base de todo lo demás

El CSS (UD4–UD5) y el JS (UD7–UD8) se apoyan en esta estructura

DATO CLAVE — Los lectores de pantalla saltan de zona en zona con una tecla. Esas zonas salen de las etiquetas HTML.

HOJA DE RUTA

Dos mitades: el esqueleto y los formularios

EL ESQUELETO DE LA PÁGINA

1. **Estructura y landmarks**
2. **Jerarquía de encabezados**

FORMULARIOS

3. **Formularios accesibles**
4. **Tipos de input**
5. **Validación nativa vs. JS**

Verificar

W3C · HeadingsMap · árbol a11y · Lighthouse

Laboratorio

La ficha de AURIA en HTML semántico

SASS

Preparar dónde irán los estilos (UD4)

CLAVE — Volveremos a esta diapositiva al pasar de una mitad a la otra.



BLOQUE 1

Estructura semántica

Qué etiqueta lleva cada zona de la página y cómo se ordenan los títulos

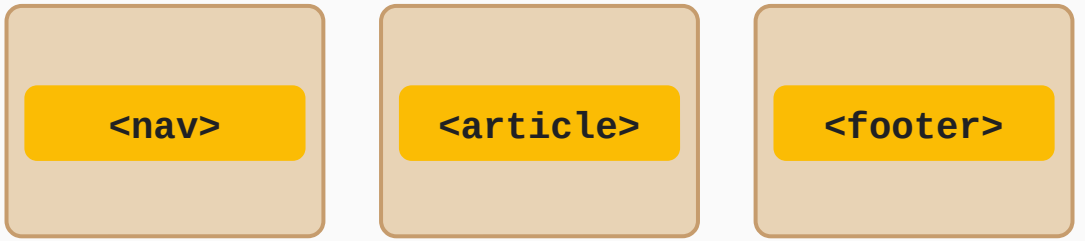
genérico vs. elemento semántico

20 cajas sin etiquetar…



¿Dónde está el menú? Hay que abrirlas todas.

…o cajas con la etiqueta puesta



Cada caja dice lo que contiene.

	genérico	Elemento semántico
Significado	Ninguno: contenedor neutro	Comunica qué es (nav, article…)
Accesibilidad	No aparece como landmark	Se anuncia como zona navegable
SEO	No aporta contexto	Ayuda a entender la jerarquía
Uso correcto	Agrupar SIN significado (p. ej. para CSS)	header, nav, main, article, section, aside, footer

CLAVE — No es una guerra contra el : es no usarlo cuando existe una etiqueta que dice exactamente lo que es.

Se ve igual… pero se oye distinto

divitis.html

```html
<div class="cabecera">
  <div class="menu">…</div>
</div>
<div class="contenido">
  <div class="ficha">…</div>
</div>
<div class="pie">…</div>
```

semantico.html

```html
<header>
  <nav>…</nav>
</header>
<main>
  <article>…</article>
</main>
<footer>…</footer>
```

El lector de pantalla anuncia:

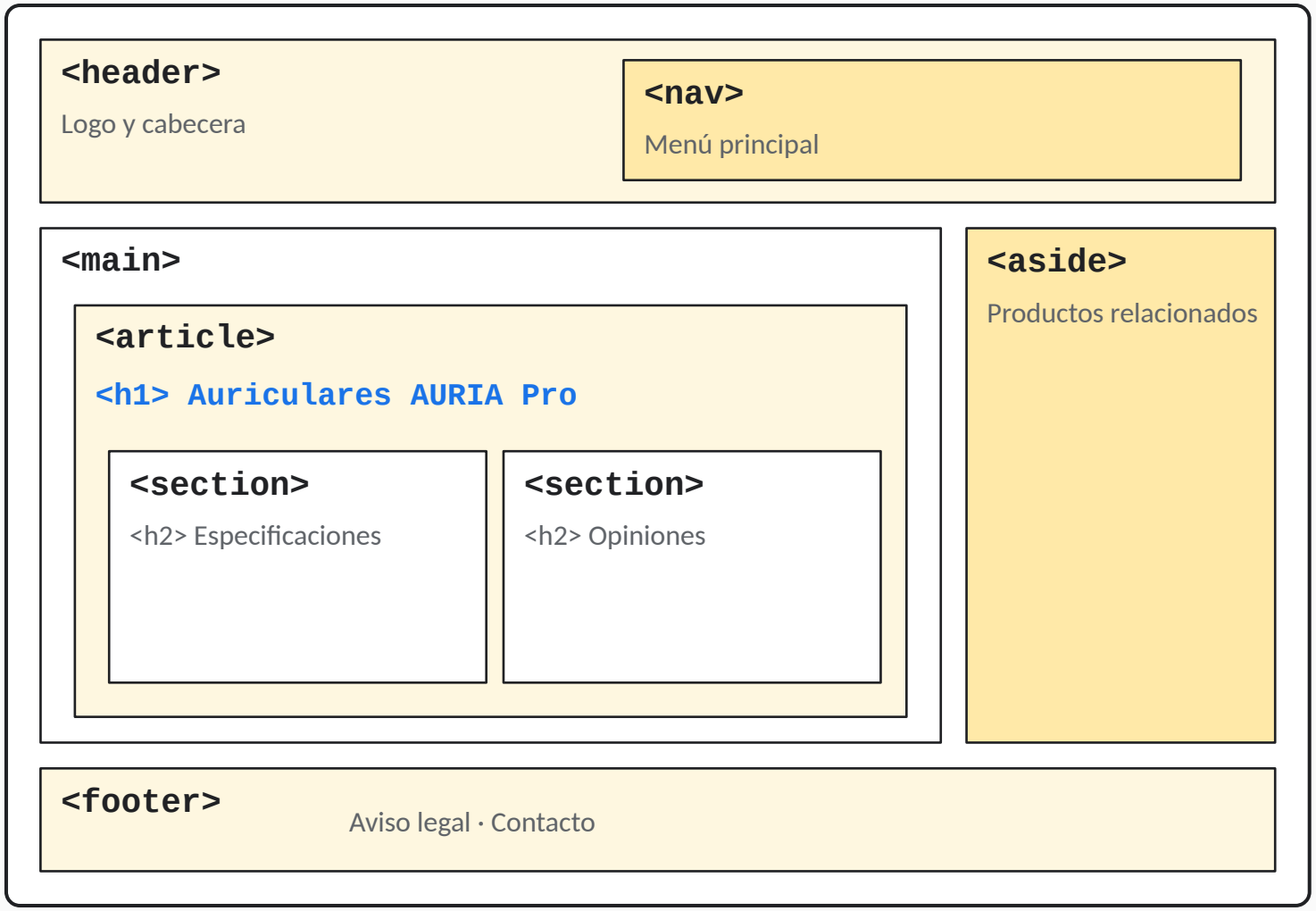
«grupo… grupo… grupo…» (o nada)

El lector de pantalla anuncia:

«banner · navegación · principal · artículo · información de contenido»

NOTA — En el navegador las dos versiones son idénticas. La diferencia se oye con un lector de pantalla y se indexa en Google.

Anatomía de una página semántica



Etiqueta → zona que anuncia el lector

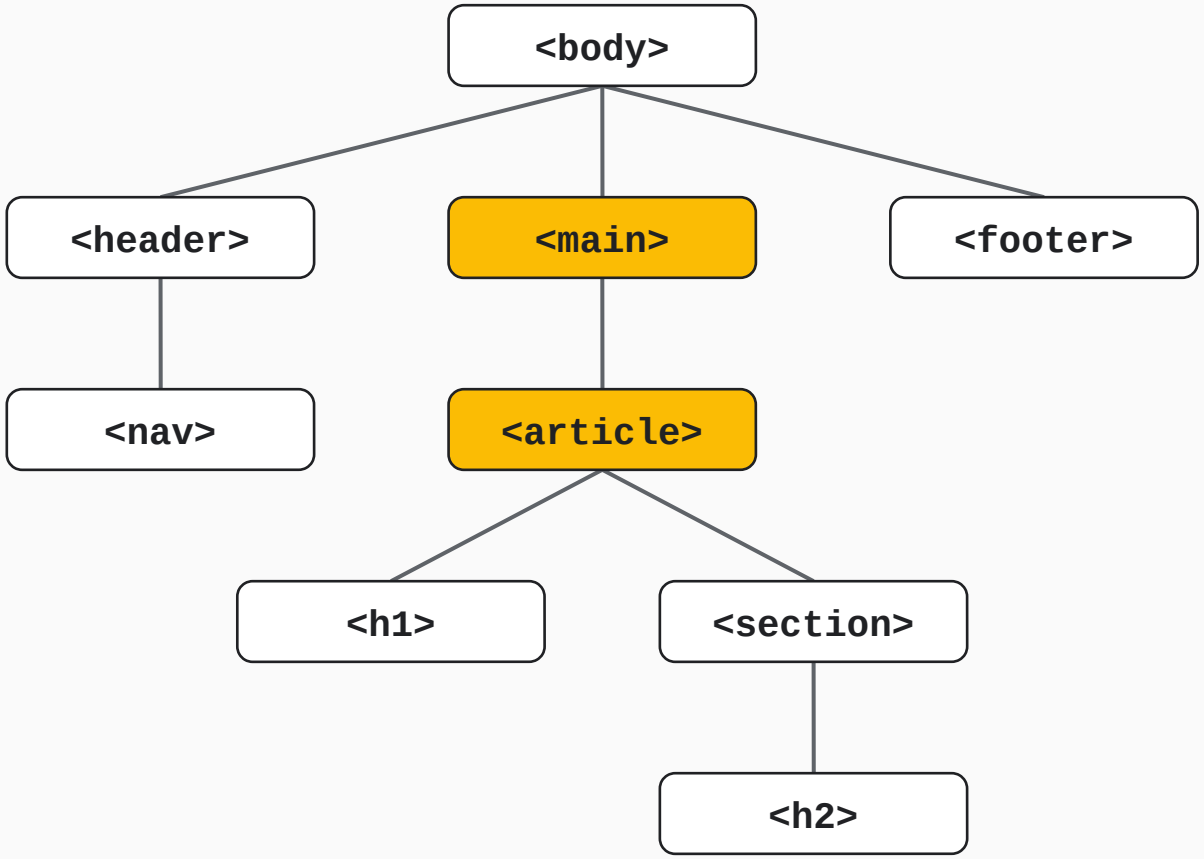
`<header>`	banner
`<nav>`	navegación
`<main>`	principal
`<aside>`	complementario
`<footer>`	inf. de contenido
`<article>`	artículo
`<section>`	región *

** solo si tiene nombre accesible (un título asociado)*

El esqueleto, en código y en árbol

auria.html

```html
<body>
  <header>
    <nav>…</nav>
  </header>
  <main>
    <article>
      <h1>Auriculares AURIA Pro</h1>
      <section>
        <h2>Especificaciones</h2>
      </section>
    </article>
  </main>
  <footer>…</footer>
</body>
```



NOTA — Cada etiqueta que se abre se cierra, y cada hija va dentro de su madre. Así es el árbol que leen el navegador y el lector de pantalla.

El catálogo de elementos de página

`<header>`

Cabecera de la página o de una sección

¿Se repite? Sí

`<nav>`

Bloque de navegación principal

¿Se repite? Sí, con moderación

`<main>`

Contenido principal de la página

¿Se repite? **NO: uno por documento**

`<article>`

Contenido autónomo y distribuible (producto, noticia, comentario)

¿Se repite? Sí

`<section>`

Bloque temático con su propio título

¿Se repite? Sí

`<aside>`

Contenido relacionado pero secundario

¿Se repite? Sí

`<footer>`

Pie de la página o de una sección

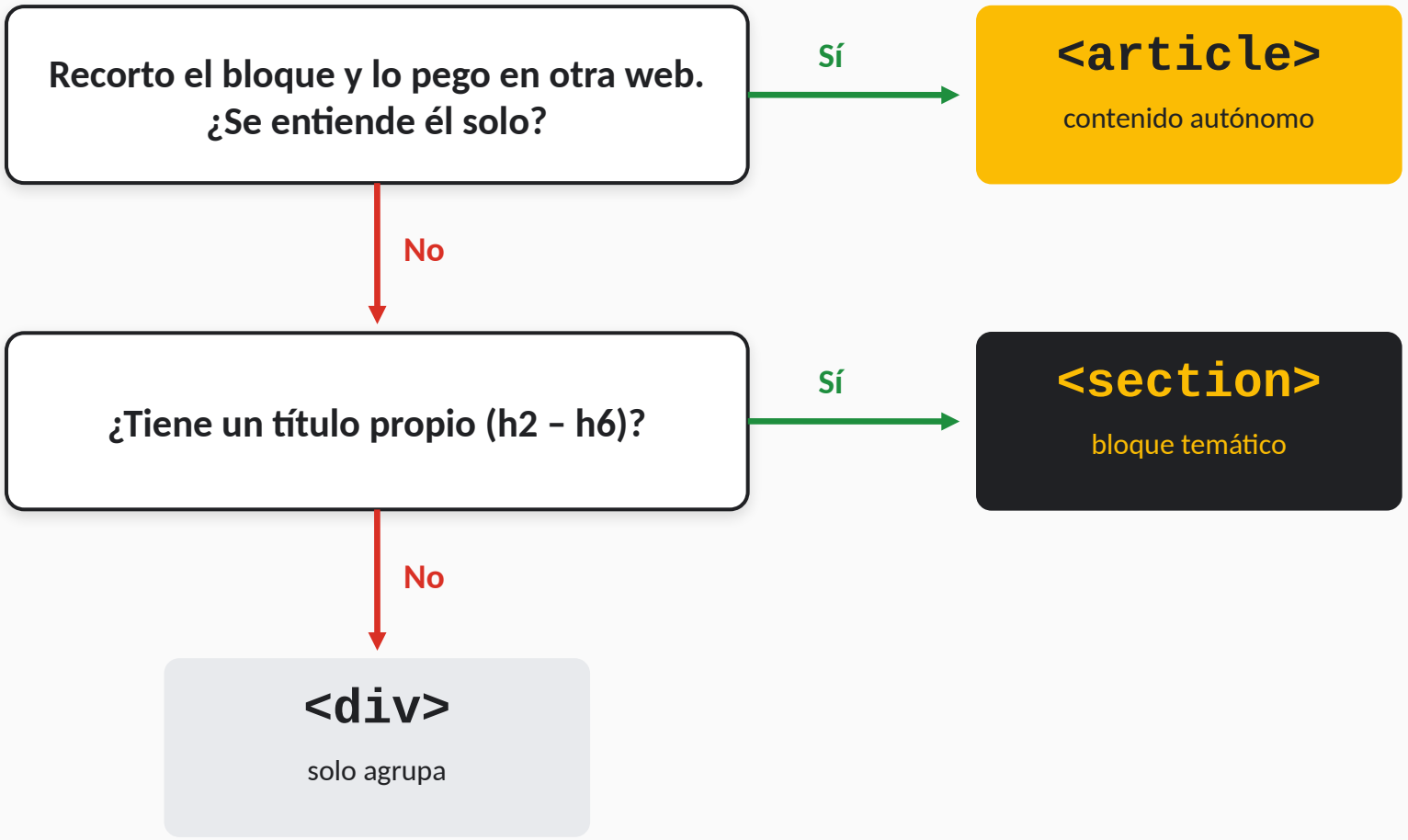
¿Se repite? Sí

`<div>`

Agrupar sin significado (para CSS)

¿Se repite? Sí: es el comodín

¿article, section o div? La prueba de recortar y pegar



Con la ficha de AURIA

Ficha completa del producto

`<article>`

En un comparador de precios se entiende sola

Bloque «Especificaciones»

`<section>`

¿Especificaciones de qué? Necesita su producto

Envoltorio para poner 2 columnas

`<div>`

No significa nada: solo es para el CSS

Del mockup de AURIA a las etiquetas



Ejercicio: ¿qué etiqueta lleva cada zona? Decidlo antes de ver las respuestas.

Los títulos son el índice del libro

Índice de un libro

Capítulo 3 · Auriculares

3.1 Especificaciones

3.1.1 Batería

3.1.2 Conectividad

3.2 Opiniones

3.2.1 Añadir reseña

Outline de la página (lo que navega el lector)

- h1 Auriculares AURIA Pro
  - h2 Especificaciones
    - h3 Batería
    - h3 Conectividad
  - h2 Opiniones
    - h3 Añadir reseña

CLAVE — h1 – h6 no son tamaños de letra: forman la estructura. El tamaño lo decide el CSS.

Tres reglas de los encabezados

1 Un solo <h1> por página

Es el título del libro: no hay tres títulos en la portada.

✓
- h1 AURIA Pro
  - h2 Especificaciones

✗
- h1 AURIA
- h1 AURIA Pro
- h1 Opiniones

2 No saltar niveles

De h1 a h2, de h2 a h3. Nunca de h1 a h3.

✓
- h1 AURIA Pro
  - h2 Opiniones
    - h3 Añadir reseña

✗
- h1 AURIA Pro
  - h3 Opiniones

¿y el h2?

3 Nivel por importancia, no por tamaño

¿Lo quieres pequeño? Eso se hace con CSS en la UD4.

✓
- h2 Opiniones
  - h3 Añadir reseña

✗
- h2 Opiniones
  - h5 «se ve pequeño»

Cómo se comprueba: HeadingsMap

HeadingsMap · periódico digital

- h1 Logo del periódico
- h1 Titular principal — 2.º h1
  - h3 Deportes — salto de nivel
- h2 Economía
  - h4 Lo más leído — salto de nivel

1. **Instala la extensión**
   HeadingsMap (Chrome / Firefox)
2. **Ábrela en cualquier página**
   Muestra el índice de títulos
3. **Busca los avisos en rojo**
   Varios h1, saltos, títulos vacíos
4. **Haz lo mismo con tu HTML**
   Objetivo: cero avisos

DATO — Hasta las webs profesionales fallan aquí. Buscad un error en un periódico real antes de mirar el vuestro.



BLOQUE 2

Formularios accesibles

Que cualquier persona sepa qué está rellenando y que el navegador compruebe los datos

Lo que ves tú y lo que oye el lector de pantalla

Texto suelto encima de la caja

Correo electrónico



«Campo de edición, vacío»

Para el lector, la cercanía visual no existe

Con <label> unido al campo

Correo electrónico



«Correo electrónico, campo de edición, obligatorio»

Además: clic en el texto → el cursor salta a la caja

PRUEBA RÁPIDA — Haz clic en cada texto de tu formulario. Si el cursor no salta a su caja, ese campo no tiene nombre.

El patrón label + input + error



campo-formulario.html

```html
<div class="campo">
  <label for="email">Correo electrónico</label>
  <input id="email" type="email" required
         aria-describedby="email-error">
  <span id="email-error" hidden>Formato no válido</span>
</div>
```

La etiqueta de la maleta

La maleta puede estar en cualquier punto de la cinta: la etiqueta dice de quién es. El for es esa etiqueta.
El solo agrupa: no significa nada.

NOTA — Sin la unión for / id, un lector de pantalla no anuncia qué campo es. Es obligatoria.

Tipos de input especializados

`type="email"`

nombre@dominio.es

Teclado con @ y valida el formato

`type="tel"`

600 123 456

Teclado numérico de teléfono

`type="number"`

4 ▲▼

Flechas y valida min / max

`type="date"`

08/10/2026 📅

Calendario nativo

`type="url"`

https://

Valida la sintaxis de una URL

`type="password"`

••••••••

Oculta lo que se escribe

`type="search"`

auriculares ×

Semántica de buscador y botón de borrar

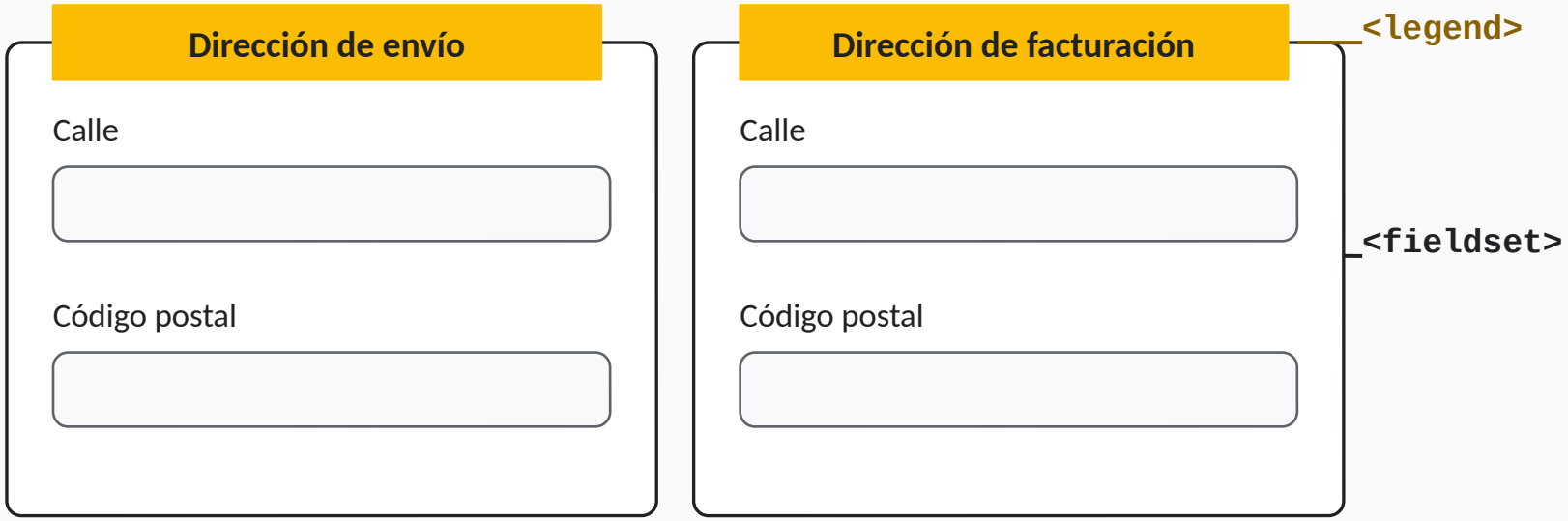
`type="checkbox"`

☑ varias ◉ una (radio)

checkbox: varias · radio: una. Accesibles por teclado

NOTA — Cambiar text por email es una palabra: a cambio, teclado adaptado en el móvil y validación sin JavaScript.

Agrupar campos con fieldset y legend



Sin fieldset: *«Calle, campo de edición»… ¿de envío o de facturación?*

Con fieldset + legend: *«Dirección de facturación, grupo. Calle, campo de edición»*

formulario.html

```html
<fieldset>
  <legend>
    Datos de contacto
  </legend>
  <label for="tel">
    Teléfono</label>
  <input id="tel"
    type="tel">
  <label for="email">
    Correo</label>
  <input id="email"
    type="email" required>
</fieldset>
```

CLAVE — El legend le pone «apellido» a cada campo del grupo. Imprescindible en formularios largos.

select y datalist: elegir entre opciones

<select>

menú cerrado

Talla

M ▼

- S
- M
- L

Solo se puede elegir una de la lista. No puedes escribir «XXXL».

select.html

```
<select id="talla">
  <option value="s">S</option>
  <option value="m">M</option>
</select>
```

<datalist>

autocompletado

Color (puedes escribir otro)

N|

- Negro

Sugiere mientras escribes, pero acepta «Verde» aunque no esté.

datalist.html

```
<input id="color" list="colores">
<datalist id="colores">
  <option value="Negro">
</datalist>
```

NOTA — Los dos llevan su <label for>. Eso no cambia nunca.

Validación: tres capas, y una es obligatoria

HTML

Hoy

1 · HTML nativo

El navegador, con atributos

`required · type · min / max · pattern`

Gratis, accesible y funciona aunque falle el JS

JS

UD8

2 · JavaScript

Reglas complejas

«las contraseñas deben coincidir»

Mejor experiencia, pero si el script falla, no hay nada

Backend

3 · Servidor

OBLIGATORIA siempre

Revalida todos los datos

Lo del navegador se puede manipular

La discoteca: la validación del navegador es el portero de la puerta (ahorra colas). La del servidor es quien te pide el DNI dentro: cualquiera puede colarse por la puerta de atrás.

DEMO — F12 → borra el atributo required del campo → envía: pasa. Por eso el servidor siempre revalida.

Validación nativa: los atributos

`required`	Campo obligatorio	`<input id="nombre" required>`	⚠ Completa este campo
`type="email"`	Forma de correo	`<input type="email">`	⚠ Incluye un signo «@»...
`min · max`	Límites de un número o fecha	`<input type="number" min="1" max="5">`	⚠ El valor debe ser ≤ 5
`minlength · maxlength`	Longitud del texto	`<textarea minlength="20">`	⚠ Alarga el texto a 20 caracteres
`pattern`	Formato con expresión regular	`<input pattern="[0-9]{5}">`	⚠ Utiliza el formato solicitado

Mensaje del navegador (Chrome, en español)

NOTA — Para un código postal: pattern="[0-9]{5}" obliga a escribir exactamente 5 cifras.

Validar nuestro propio HTML



Resultado del validador

Error End tag main seen, but there were open elements.
From line 9

Error Duplicate ID email.
From line 14

Terminal

```
# Alternativa desde terminal (Node)
npx html-validate estructura.html
```

NOTA — Un HTML con errores puede verse bien: el navegador cierra etiquetas por su cuenta, y el lector de pantalla puede entender otra cosa.



BLOQUE 3

Manos a la obra

Laboratorio AURIA, herramientas de verificación y casos reales

LABORATORIO

Traducir la ficha de AURIA a HTML semántico

1

Estructurar la página

header, nav, main y footer

2

article o section

Justificado para el bloque del producto

3

Formulario «Añadir reseña»

fieldset, legend y labels con for / id

4

3 tipos de input

Distintos de text: email, date, number…

5

Verificar

HeadingsMap y validador W3C

Ni una línea de CSS. Ni style="". Hoy solo importa qué es cada cosa, no cómo se ve.

ÉXITO — Cero saltos de nivel en HeadingsMap y cero errores en el validador del W3C.

Cuatro herramientas, cuatro preguntas

Validador W3C

¿Es HTML válido?

validator.w3.org

→

HeadingsMap

¿Tiene sentido el índice de títulos?

Extensión del navegador

→

Árbol de accesibilidad

¿Qué «ve» el lector de pantalla?

F12 → Elements → Accessibility

→

Lighthouse

Primera nota de accesibilidad (0–100)

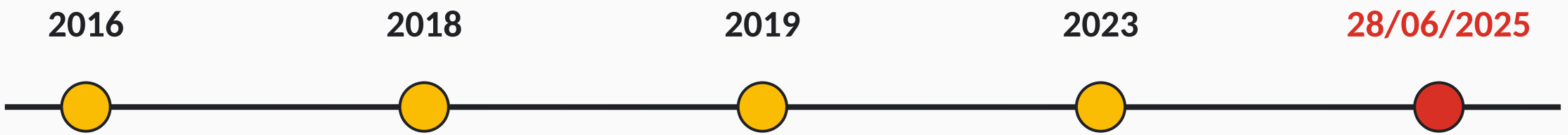
F12 → Lighthouse → Accesibilidad

```
Árbol de accesibilidad:  navigation “Principal”   main   article   heading “Auriculares AURIA Pro” level 1
generic  ← un <div>
```

OJO — Lighthouse es una primera aproximación, no un certificado: un 100 no garantiza que la página sea accesible (UD9 – UD10).

CASO REAL

HTML semántico en proyectos reales: es ley



Directiva (UE) 2016/2102

Accesibilidad de webs y apps del sector público

RD 1112/2018

España: obligatorio para todas las administraciones

Acta Europea de Accesibilidad

Directiva (UE) 2019/882: productos y servicios privados

Ley 11/2023

España la incorpora a su legislación

Ya es obligatoria

Comercio electrónico, banca, transporte… (salvo microempresas)

React y Vue generan HTML

Si el componente es todo , el resultado también.

Code review

Revisar la estructura semántica es parte normal de la revisión.

Auditorías automáticas

Lighthouse y axe miran la semántica antes que el CSS.

AURIA — Es una tienda online: si no es microempresa, desde 2025 su web tiene que ser accesible por ley.

ERRORES COMUNES (1/2)

Trampas de estructura y encabezados

«Divitis»: todo son

Rompe la navegación por landmarks

```
<div class="menu">…</div>
```

```
<nav>…</nav>
```

Varios <h1> en la página

Confunde el outline: debe haber uno

```
<h1>AURIA</h1> … <h1>Opiniones</h1>
```

```
<h1>AURIA Pro</h1> … <h2>Opiniones</h2>
```

Saltar niveles de encabezado

h1 → h3 rompe la jerarquía navegable

```
<h1>…</h1> <h3>Batería</h3>
```

```
<h1>…</h1> <h2>Batería</h2>
```

Elegir la etiqueta por el tamaño

El tamaño es cosa del CSS

```
<h5>Opiniones</h5>  «se ve pequeño»
```

```
<h2>Opiniones</h2> + CSS en la UD4
```

ERRORES COMUNES (2/2)

Trampas de formularios

input sin label asociado

El lector solo dice «campo de edición»

❌
```
<p>Correo</p> <input id="email">
```

✅
```
<label for="email">Correo</label>
```

Placeholder en vez de label

Desaparece al escribir y tiene poco contraste

❌
```
<input placeholder="Correo">
```

✅
```
<label for="email">Correo</label> + placeholder
```

Validar solo con JavaScript

Si el script falla, no hay validación

❌
```
if (email === '') { … }  // y nada más
```

✅
```
<input type="email" required>
```

Obligatorios sin required

El usuario se entera al enviar

❌
```
<label>Nombre *</label> <input id="n">
```

✅
```
<input id="n" required>
```

CONTROL DE COMPRENSIÓN

Preguntas de control y debate

1. ¿Por qué un nunca aparece en la lista de landmarks de un lector de pantalla?
2. ¿Qué problema provoca saltar de <h1> a <h3> sin pasar por <h2>?
3. ¿Por qué label e input deben unirse con for / id y no basta con ponerlos cerca?
4. ¿Qué aporta fieldset / legend que no aporte agrupar con un div?
5. ¿Por qué la validación nativa de HTML no sustituye a la validación en el servidor?

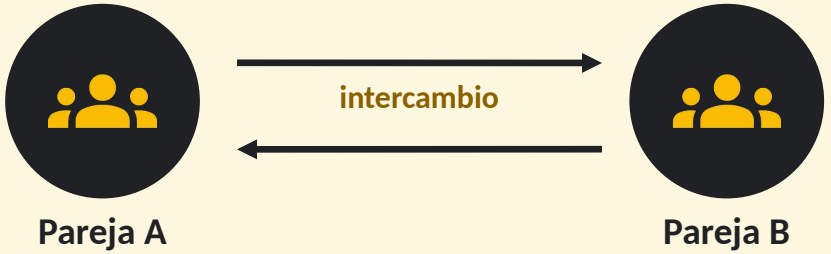
MINI-RETO PRÁCTICO

Reto express: 20 minutos, en parejas

1	**Landmarks**	header, nav, main, footer
2	**Outline**	Sin saltos de nivel
3	**Formulario**	≥ 4 campos, ≥ 2 tipos de input
4	**Labels**	for / id en todos los campos
5	**Validador**	Cero errores

Elegid un formulario de la vida real

Gimnasio · cita médica · reserva de mesa · torneo



- HeadingsMap
- Validador W3C
- Clic en cada texto → ¿salta a su caja?

ÉXITO — Hoy los revisores sois vosotros: validador sin errores y outline navegable en el trabajo de la otra pareja.

GLOSARIO · HTML

Las palabras de la unidad

HTML semántico	Elegir cada etiqueta por lo que el contenido ES, no por cómo se ve
divitis	Usar para todo, aunque exista una etiqueta con significado
landmark	Zona de la página que el lector anuncia y a la que se puede saltar
rol ARIA	Lo que un elemento «es» para la accesibilidad: <nav> = navigation
árbol a11y	Versión de la página que el navegador entrega al lector de pantalla
nombre accesible	Texto con el que se anuncia un elemento (p. ej. su <label>)
lector de pantalla	Programa que lee la página en voz alta: NVDA, JAWS, VoiceOver
outline	Índice de la página formado por los encabezados h1 – h6
for / id	Une un <label> con su campo: mismo valor en los dos atributos
fieldset / legend	Agrupa campos relacionados y les da un título común
aria-describedby	Asocia a un campo su texto de ayuda o de error
validación nativa	Comprobaciones del navegador con atributos, sin JavaScript
pattern	Expresión regular que debe cumplir todo el valor del campo
W3C Validator	Comprueba que el HTML cumple la especificación



BLOQUE 4

Preparando los estilos: SASS

Dónde y cómo vamos a escribir el CSS de AURIA en la UD4

¿Qué es SASS? Un preprocesador de CSS



La receta y el plato

El .scss es tu receta, con tus abreviaturas («salsa base ×2»). El .css es el plato servido: el cliente (el navegador) nunca ve la receta.

Nunca edites el CSS generado

Lo genera Vite desde el .scss: en la siguiente compilación tu cambio desaparece. Todo el CSS que ya sabes es SCSS válido.

CLAVE — El navegador no entiende SASS: alguien tiene que compilarlo. En nuestro proyecto lo hace Vite, dentro de Docker.

Instalar y compilar: Vite + Sass en Docker

compose.yaml

```yaml
services:
  web:
    image: node:22-alpine
    user: node
    working_dir: /app
    volumes: [".:/app"]
    ports: ["5173:5173"]
    command: npx vite --host 0.0.0.0
```

Terminal

```
# 1. Ficha del proyecto (solo la primera vez)
docker compose run --rm web npm init -y
# 2. Vite y Dart Sass como herramientas de desarrollo
docker compose run --rm web npm install -D vite sass
# 3. Arrancar Vite y dejarlo vigilando
docker compose up
```

Enlazas el .scss

<link href="/scss/main.scss">: Vite lo compila al servirlo.

localhost:5173

Guardas el .scss y el navegador se actualiza solo (HMR).

Source map

css.devSourcemap en vite.config.mjs: F12 te dice qué .scss y qué línea.

NOTA — No instalas Node en el PC: npm, Vite y Sass viven en el contenedor. Deja docker compose up abierto: los errores de SASS salen ahí.

Variables: los tokens de la UD2, en código

_tokens.scss

```scss
// Nivel 1 · Global
$blue-500: #1E88E5;
$space-md: 16px;

// Nivel 2 · Semántico
$color-brand-primary: $blue-500;
```

compila

main.css

```css
.button {
  background: #1E88E5;
}

/* las $variables desaparecen:
   solo queda el valor */
```

	Variable SASS $color	Custom property CSS --color
¿Cuándo existe?	Solo al compilar	En el navegador, con la página abierta
¿Cambia en vivo?	No	Sí: modo oscuro, JavaScript, media queries
Uso típico	Paleta, breakpoints, cálculos	Tokens semánticos que cambian con el tema

CLAVE — No se sustituyen, se complementan: los valores en SASS y los tokens del tema publicados como --custom-properties.

Anidamiento: el CSS sigue la forma del HTML

_card.scss

```scss
.card {
  padding: $space-md;

  &:hover {        // .card:hover
    box-shadow: …;
  }
  &__title {       // .card__title
    font-size: 1.25rem;
  }
}
```

main.css

```css
.card { padding: 16px; }
.card:hover { box-shadow: …; }
.card__title { font-size: 1.25rem; }

/* & = «el selector en el que estoy» */
/* card__title: nombre BEM (UD5) */
```

Error n.º 1: anidarlo todo

`.page { main { article { section { h2 {…} } } } }` → `.page main article section h2`

Selector larguísimo, muy específico y que se rompe al cambiar el HTML.

REGLA — Como mucho 3 niveles. Anida sobre todo &:hover, &:focus, &__elemento y &--modificador.

Parciales y @use: un fichero por pieza

```
proyecto/
    compose.yaml    ← Docker
    package.json · vite.config.mjs
    index.html    ← enlaza scss/main.scss
    dist/    ← generado, NO se toca
    scss/
        _tokens.scss    ← UD2
        _mixins.scss
        _base.scss
        components/
            _button.scss
            _card.scss
        main.scss    ← solo @use
```

main.scss

```
// el índice: no lleva estilos propios
@use 'base';
@use 'components/button';
@use 'components/card';
```

components/_card.scss

```
@use '../tokens' as *;   // ../ = una carpeta arriba
.card { padding: $space-md; }
```

`_` El guion bajo = parcial: no se compila solo, es una pieza de otro fichero

`@import` Obsoleto desde Dart Sass 1.80; se elimina en Dart Sass 3.0. Usa @use

ERROR TÍPICO — «Can't find stylesheet to import»: casi siempre es la ruta del @use, que es relativa al fichero que lo escribe.

Mixins y funciones: reutilizar y calcular

_mixins.scss

```scss
@use 'tokens' as *;

@mixin flex-center($gap: 0) {
  display: flex;
  justify-content: center;
  align-items: center;
  gap: $gap;
}
```

components/_button.scss

```scss
@use 'sass:color';
@use '../tokens' as *;
@use '../mixins' as *;
.button {
  @include flex-center($space-md);
  background: $color-brand-primary;
  &:hover {
    background: color.adjust(
      $color-brand-primary, $lightness: -10%);
  }
}
```

Repites un valor → **variable**

Repites un grupo de líneas → **mixin**

No repites nada → **CSS normal**

color.adjust: el hover depende del color de marca

$color-brand-primary | **hover (−10 % luz)**

Si cambia el azul, el hover se recalcula solo.

NOTA — No conviertas en mixin algo que usas una sola vez.

BLOQUE 4 · SASS

Errores frecuentes con SASS

Editar el CSS generado a mano

Se pierde en la siguiente compilación

✗ `Cambio el color en dist/assets/…css`

✓ `Cambio la variable en scss/_tokens.scss`

Abrir el HTML con doble clic

Sin Vite nadie compila el .scss: sale sin estilos

✗ `file:///…/index.html`

✓ `http://localhost:5173`

Rutas mal en @use

Can't find stylesheet to import

```
✗ @use 'tokens';  // desde components/

✓ @use '../tokens' as *;
```

Contenedor parado

«He cambiado el .scss y no se ve nada»

✗ `Guardo el .scss sin Vite en marcha`

✓ `docker compose up (y mira sus errores)`

GLOSARIO · SASS

Las palabras de SASS

`preprocesador`	Lenguaje que se traduce a CSS antes de llegar al navegador
`SCSS`	Sintaxis de SASS con llaves; todo CSS válido es SCSS válido
`compilar`	Traducir los .scss a un .css que entiende el navegador
`Dart Sass`	Compilador oficial (paquete sass). Node Sass está abandonado
`Vite`	Servidor de desarrollo: compila al guardar y recarga solo (HMR)
`source map`	Fichero .map que enlaza el CSS compilado con su .scss
`$variable`	Valor reutilizable que solo existe al compilar
`--custom-prop`	Variable de CSS que vive en el navegador y puede cambiar
`& (padre)`	El selector en el que estoy: &:hover → .card:hover
`_parcial.scss`	Pieza que no se compila sola; se carga desde otro fichero
`@use`	Carga un módulo una vez, con espacio de nombres (tokens.$x)
`@mixin / @include`	Define un bloque de declaraciones / lo inserta
`función`	Devuelve un valor: color.adjust, math.div, @function propia
`@import`	Sistema antiguo, obsoleto: se elimina en Dart Sass 3.0

Ejercicio: la arquitectura SASS de AURIA

1

Instalar y estructurar

Vite + Sass en Docker y la carpeta scss/ con sus parciales

2

Tokens a variables

Los de Penpot (UD2): global y semántico

3

Mixin y botón

flex-center + _button.scss con hover calculado

4

Arrancar y enlazar

docker compose up y <link> a scss/main.scss

5

Prueba de fuego

Cambia solo $blue-500: cambian botón, hover y foco

Sobre la carpeta de la ficha de AURIA: el HTML de hoy no se toca, solo se prepara dónde irán sus estilos en la UD4.

ÉXITO — Al cambiar una sola variable cambian el botón, su hover y el contorno de foco. Si tienes que tocar algo más, algo está mal.

ÚLTIMA REFLEXIÓN

¿Por qué revisar el HTML antes del CSS ahorra tiempo?

JavaScript

UD7 – UD8 · busca elementos por su estructura

CSS

UD4 – UD5 · apunta a la estructura (nav a, article h1…)

HTML semántico

UD3 · los cimientos

Arreglar los cimientos al principio es barato.

Arreglarlos con el edificio encima es carísimo.

Igual que con los tokens de la UD2: el ahorro se nota el día que hay que cambiar algo.

DEBATE — Antes de ver la respuesta: ¿qué se rompe en el CSS si mañana quitáis un <section>?

CHECKPOINT

Resumen de puntos clave

- Cada etiqueta HTML5 comunica significado a navegadores, buscadores y lectores de pantalla
- Un formulario accesible empieza por unir cada label con su input, no por el estilo
- Los títulos forman un índice: un solo h1 y sin saltos de nivel
- fieldset y legend dan contexto cuando hay varios campos relacionados
- Validación nativa para la experiencia; la del servidor, siempre
- SASS organiza el CSS (variables, parciales, mixins) y se compila a un .css normal

SIGUIENTE · UD4

CSS3 moderno

- Cascada y especificidad
- Flexbox y Grid
- Responsive y Container Queries
- Escrito ya en SASS

AURIA ya está en HTML, fea pero correcta, y con su arquitectura SASS lista. En la UD4 le ponemos la piel.